

DESCRIPCIÓN GENERAL ESTRUCTURA

Aquí tienes un análisis estructurado de las similitudes y diferencias entre los tres documentos, seguido de una guía unificada y lista para usar que consolida lo mejor de cada enfoque, alineada con la convención oficial de OpenCode y prácticas de desarrollo asistido por IA (2026).

Análisis Comparativo

Aspecto	Documento 1	Documento 2	Documento 3
Archivos base	<code>system.md</code> , <code>rules.md</code> , <code>context.md</code> , <code>agents.md</code> , <code>memory.md</code> (recomendado)	<code>system.md</code> , <code>rules.md</code> , <code>context.md</code> , <code>agents.md</code>	<code>system.md</code> , <code>rules.md</code> , <code>context.md</code> , <code>agents.md</code>
Enfoque principal	Comportamiento por archivo + ejemplo funcional completo (<code>e-commerce-api</code>)	Función técnica detallada + estructura de proyecto estándar + archivos opcionales de negocio/docs	Convención oficial OpenCode + flujo de carga interno + ejemplos listos para copiar
Orden de carga	Se carga al inicio de sesión; <code>system.md</code> es la capa superior	No especifica orden; <code>system.md</code> tiene prioridad máxima ante conflictos	Explicita concatenación: <code>rules.md</code> → <code>context.md</code> → <code>agents.md</code> → <code>system.md</code>
Archivos opcionales sugeridos	<code>memory.md</code> , <code>glossary.md</code> , <code>decisions.md</code> /a <code>dr/</code> , <code>playbooks.md</code> , <code>CONTRIBUTING.m</code> <code>d</code> , <code>README.md</code>	Igual que Doc 1	<code>tools.md</code> , <code>examples.md</code> , <code>workflows.md</code>

Gestión de conflictos	<code>system.md</code> manda sobre todo	<code>system.md</code> es la capa más alta; si algo lo contradice, prevalece	Proximidad al prompt da peso, pero se recomienda coherencia absoluta; <code>rules.md</code> actúa como hard constraint
Estilo/Formato	Técnico, estructurado, ejemplo completo	Descriptivo, centrado en función y contenido	Tablas, emojis, checklist de implementación, ejemplos copiables

✓ Similitudes Clave

- Los 4 archivos base (`system.md`, `rules.md`, `context.md`, `agents.md`) son el núcleo inamovible.
- Todos se alinean con OpenCode y herramientas de agentes IA modernas.
- `system.md` define identidad y flujo; `rules.md` son límites duros; `context.md` es referencia técnica; `agents.md` son sub-roles o "sombreros" especializados.
- Se enfatiza la no exposición de `.env`, la validación con tests/lint, y la coherencia técnica para evitar que la IA "invente" stack o rutas.

⚠ Diferencias a Resolver

- Orden de inyección vs. prioridad lógica.
- Qué archivos opcionales priorizar según el caso de uso.
- Nivel de detalle en ejemplos y flujos de validación.



Guía Unificada: Estructura Estándar `.ai/` para Desarrollo Asistido por IA

Esta guía consolida las mejores prácticas de las tres referencias, resolviendo contradicciones y presentando un estándar reproducible, priorizado y compatible con OpenCode y agentes LLM modernos.

Estructura Base del Proyecto

text

mi-proyecto/

```
|— .ai/
|   |— system.md    # 🧠 Prompt de sistema (comportamiento global)
|   |— rules.md     # ⚖️ Reglas obligatorias (seguridad, calidad, límites)
|   |— context.md   # 📖 Contexto técnico (stack, arquitectura, convenciones)
|   |— agents.md    # 🤖 Catálogo de agentes especialistas (sub-roles)
|   |— [opcionales] # memory.md, tools.md, workflows.md, etc.
|— .env            # 🔒 Variables sensibles (ignoradas por Git)
|— .gitignore
|— opencode.json   # ⚙️ Config del runtime/agente (modelo, herramientas,
rutas)
|— .vscode/settings.json # (Opcional) Formateo, lint, paths del editor
|— src/           # 💻 Código fuente del producto
```

Función, Prioridad y Contenido de Cada Archivo

Archivo	Nivel	Función	Qué debe contener
<code>system.md</code>	🔴 Alta (Núcleo)	Define identidad, tono, flujo de trabajo y formato de salida. Es el "manual de comportamiento".	Rol del asistente, estilo de respuesta (pasos, idioma, estructura), referencias a los otros archivos, restricciones base, manejo de incertidumbre ("preguntar antes de asumir").

<code>rules.md</code>	● Alta (Filtro)	Reglas innegociables de seguridad, calidad, Git y alcance. Es el "código penal".	Prohibición de exponer <code>.env/secrets</code> , convenciones de naming/lint, requisitos de testing, políticas de commits, manejo de errores, límites de modificación (ej: solo <code>/src</code>).
<code>context.md</code>	● Media (Contexto)	"Memoria técnica" estable. Documentación de referencia, no reglas.	Stack y versiones, estructura de carpetas, esquema DB/tablas, endpoints clave, servicios externos, decisiones técnicas resumidas (ADRs), cómo correr/build/test.
<code>agents.md</code>	● Baja/Media	Define sub-roles o "sombreros" especializados. No son procesos separados.	Lista de agentes, triggers de activación, capacidades, límites, checklist de salida por agente (ej: QA siempre sugiere casos de prueba).

⚙️ Flujo de Ejecución y Resolución de Conflictos

1. Carga: OpenCode (y herramientas compatibles) ensambla los archivos en un único prompt de sistema antes de cada interacción. El orden típico de concatenación es:

`rules.md → context.md → agents.md → system.md`

2. Prioridad lógica: Aunque `system.md` se inyecta al final (y por proximidad tiene mayor peso contextual), debe actuar como coordinador, no como excepción a las reglas. `rules.md` funciona como *hard constraint*: si una petición lo viola, el agente debe alertar y proponer alternativas compatibles.
3. Coherencia cruzada: Mantén alineación absoluta. Ejemplo: si `rules.md` prohíbe `any` en TypeScript, `system.md` no debe sugerir flexibilidad, sino reforzar el uso de tipos estrictos o `unknown` con validación.
4. Agentes: Se activan por triggers semánticos en el prompt del usuario. Heredan `system.md`, pero restringen su alcance según `agents.md`.

Archivos Opcionales Recomendados (Según Caso de Uso)

Archivo	Cuándo añadirlo	Función principal
<code>memory.md</code>	Proyectos grandes o de larga vida	Historial de decisiones, patrones probados, errores frecuentes a evitar. Evita retrocesos y discusiones repetidas.
<code>tools.md</code>	Agente con capacidad de ejecutar comandos	Lista de herramientas permitidas/prohibidas (<code>npm run</code> , <code>prisma</code> , <code>git</code> , <code>docker</code> , <code>curl</code>) y sintaxis de invocación.
<code>workflows.md</code>	Tareas repetitivas (deploy, migraciones, PRs)	Runbooks paso a paso. El agente los sigue literalmente para estandarizar procesos operativos.
<code>examples.md</code>	Errores recurrentes de formato o output	Ejemplos de respuestas correctas/incorrectas, plantillas de código, patrones de validación.

<code>glossary.md / decisions.md</code>	Equipos grandes o dominios complejos	Términos de negocio unificados y registro de Decisiones Arquitectónicas (ADRs).
---	--------------------------------------	---

✓ Checklist de Implementación Rápida

- Crear carpeta `.ai/` y los 4 archivos base.
- Redactar `system.md` primero (define tono, flujo y referencias).
- Configurar `rules.md` como hard constraints (seguridad + calidad + Git).
- Documentar `context.md` con hechos técnicos verificables y actualizados.
- Definir `agents.md` solo si necesitas especialización por tipo de tarea.
- Validar coherencia cruzada (sin contradicciones internas).
- Probar carga: ejecutar `opencode "Resume tus 3 reglas más importantes"` o equivalente para verificar inyección correcta.
- Commit inicial: `chore(ai): initialize .ai/ folder with standard OpenCode config`

💡 Nota final: Esta estructura es agnóstica al framework pero nativa a OpenCode y compatible con la mayoría de IDEs con integración de agentes LLM. Puedes escalarla progresivamente: empieza con los 4 archivos base, añade `tools.md` o `workflows.md` cuando el agente necesite ejecutar comandos, y usa `memory.md` o `decisions.md` cuando el proyecto supere los ~6 meses de vida activa.